# DeepSeek Harness 拆解：一套能拼装的 Agent 架构

原创 腾讯程序员 腾讯技术工程 2026年8月14日 02:20 广东 1086人



作者：chino，腾讯WXG 微信小店前端开发工程师

DeepSeek 发布了第一款 Agent 产品，DeepSeek Harness。这次的重点放在了运行时架构上，官方给的关键词是"everything is a plugin"，模型接入、工具执行、会话记录、循环本身、界面，全部走插件机制。




这篇文章跳过 agent 领域已经成为共识的那部分——检测工具调用、执行、回填、循环，这套流程眼下几乎每个框架都长一个样，不值得再讲一遍。重点放在 DeepSeek Harness 真正跟别人不一样的几处设计：Cordis 这套插件运行时具体怎么管生命周期，preset 的 scope 继承链为什么拆成两层，Code Mode 为什么选了一个听起来不太"安全"的隔离方案，工具注册的遮蔽算法怎么写。顺带会跟 Codex 的几个具体工程决策做对比，只挑不一样的地方讲。

## 一、为什么是 Cordis

"everything is a plugin" 的基础是 Shigma 的开源项目 Cordis。定位是"Meta-Framework for Modern JavaScript Applications"，一个通用的插件框架，处理依赖注入、作用域服务、生命周期清理这几件事，与 agent、与 LLM 都没有直接关系，在 DeepSeek Harness 出现之前就已经存在。

Cordis 在开源社区有实际的使用者。最典型的是 Koishi，一个跨平台聊天机器人框架（QQ / Discord / Telegram / 微信都接）。聊天机器人场景天然是"几十个插件拼在一起、热插拔、随时改配置"，与 agent 的场景几乎相同，只是没有 LLM。DeepSeek Harness 把整个框架的源码 vendor 进自己的仓库，改了个 scope 叫 `@deepseek-ai/cordis`，然后

把公司自己写的每一个包，都设成对它的 peer dependency。这比"使用一个第三方库"更进一步：整个产品都构建在 cordis 之上。

先对比几种常见的插件形态：传统 DI 容器、轻量钩子方案、Cordis。这三种方案各有取舍，值得摆在一起看。

**传统 DI 容器的三个缺口。** "主流"方案是一个 DI 容器加生命周期注解。你 bind 了一个 service，谁来负责 unbind 和 cleanup？传统 DI 要么不管（泄漏），要么要求你手写 lifecycle hook。LLM provider 热替换了，依赖它的 ToolRegistry 应该自动重启，传统 DI 做不到这种"依赖驱动的重载"。配置上，传统 DI 的配置在启动时读一次；而 `cordis.yml` 里一行就是一个插件实例，改配置直接触发 HMR。

Pi也是Agent界的明星产品了，但**Pi 的 Extension 走的是另一条路。** Pi 的自我定位是 "self extensible coding agent"，它的插件叫 Extension（扩展），基于 TS，在扩展点上通过钩子塞逻辑：没有依赖注入，没有插件间依赖管理（加载顺序即优先级），也不能卸载。Cordis 的插件是带依赖声明、生命周期、可逆副作用的"组件"：有依赖注入，有反应性 coeffects，插件间可以声明依赖，卸载时能完整逆转副作用。

这最后一点——"可逆副作用"——是整篇文章的地基。Cordis 的设计论文把这件事上升到理论层面，核心问题问得很直接：

> 有没有一种 programming model，能让动态本身具备类似进程那样的生命周期隔离？

进程和容器的好处在于：kill 掉再启动，状态就清空了。代价是重启会丢掉进程内的 cache、connection、partial computation。插件系统希望不用重启，也能完成同样的清理。论文为此给了两个定义：

- **时间可组合性**
  ：卸载组件时，组件对共享环境所做的修改必须被完整、安全地逆转——这要求追踪组件执行的每一次资源分配、事件注册和状态变更。
- **空间可组合性**
  ：依赖变化时，相关组件自动激活或停用。

时间可组合性是这篇文章的主线。剩下的事情——Fiber、effect、系统边界、preset、Code Mode——都在回答同一个问题：怎么在工程上实现"卸载 = 完整逆转"。

Cordis 生态还带了几个配套包，随项目一起 vendor 进来：一个配置文件加载器（负责解析 `cordis.yml` 这类声明式配置），一个 include 插件（负责把一份配置当成子树挂载进来，preset 机制用的就是它改名后的版本），一个 HMR 模块（支持插件热替换），外加日志和定时器这两个基础设施插件。这几块拼起来，才能真正理解"everything is a plugin"。

## 二、Fiber 与 effect：可逆副作用的地基

Cordis 里，插件的类型定义是一个联合类型：

```
type Plugin<T> = Plugin.Function<T> | Plugin.Constructor<T> | Plugin.Object<T>
```

裸函数、类、带 `apply` 方法的对象，三种写法最终都被解析成一个统一的回调。挂载一个插件调用 `ctx.plugin(plugin, config)`，内部先找有没有已经存在的 Runtime 记录（按回调函数身份做 key，同一个插件多次挂载共享同一条 Runtime），没有就新建，再造一个 Fiber，塞进这条 Runtime 的 fibers 列表里。

Fiber 是插件实例的生命周期状态机，六个状态：`PENDING`、`LOADING`、`ACTIVE`、`FAILED`、`DISPOSED`、`UNLOADING`。插件声明了 `inject: ['tools', 'shell']`，对应的 Fiber 会停在 `PENDING`，直到这两个服务都在依赖链上出现，才真正跑这个插件的代码，进入 `ACTIVE`。这套等待机制是 Cordis 自己实现的依赖解析，插件作者不需要手写"等对方准备好"的轮询逻辑。这对应空间可组合性：依赖出现后，组件自动激活。

卸载一个插件是否安全，由 effect 机制决定。插件注册的任何东西——事件监听、服务、定时器——都通过 `ctx.effect()` 登记。登记时立刻执行一次，返回的撤销函数被推进一个 disposables 列表。论文 5.1.1 指出：Cordis 中每一次上下文变更都通过唯一原语 `ctx.effect` 完成——提供服务、实例化组件、所有修改上下文的操作都归约成一次 `ctx.effect` 调用。用法长这样：

```
ctx.effect(() => {
  // 做副作用：注册监听、开定时器、提供服务……
  return () => { /* 逆：撤销上面的操作 */ }   // 返回 disposer
})
```

callback 可以返回一个 disposer 函数、一个 Promise，或者一个（异步）迭代器，逐段 yield disposer——每做一步副作用就交一个撤销函数。

在这里，"副作用"的定义是：任何通过 Context 对共享环境进行的修改。这张表把日常操作都覆盖了：

| 类别 | 例子 |
| --- | --- |
| 事件注册 | `ctx.on("foo", handler)`<br>—— 往共享事件总线挂监听器 |
| 提供服务 | `ctx.provide(name, service)`<br>—— 往上下文上"放"一个服务 |
| 挂载子组件 | `ctx.plugin(plugin)`<br>—— 子插件本身就是一个"父级上的副作用" |
| 上下文扩展 | `ctx.extend()`<br>/ `ctx.intercept()` / `ctx.isolate()` |
| 外部资源 | 定时器、文件监听、HTTP server、子进程、DB 连接等 |
| 状态变更 | 改 config、改注册表、改 store |

Koishi 里 `ctx.command(...)` 注册命令、`ctx.router.get(...)` 注册 HTTP 路由；DSH 里注册工具、挂 HMR 监听、起调度器——全是副作用。"怎么逆转"，实现上很直接：每个副作用都带着自己的撤销函数，运行时把它们按 LIFO 叠起来，卸载时整体执行一遍：

```
disposables.splice(0).reverse().forEach(dispose => dispose())
```

按注册的相反顺序逐个撤销，类似退栈。子 Fiber 的销毁函数本身也是通过 `parent.fiber.effect(...)` 挂在父 Fiber 上注册的——在源码里就是 Fiber 构造函数末尾这一行：

```
this.dispose = parent.fiber.effect(() => { /* 注册插件 */
  return async () => { /* 注销插件 */ }
})
```

也就是说，一个插件本身 = 父 fiber 上的一个 effect：插件的启动逻辑就是副作用，卸载逻辑就是逆。子组件的逆会被 prepend 到父上下文的累加器上，形成递归结构（论文里叫 twisted composition，记作 $\partial^2\Gamma$）。所以卸载一个插件，会连带把它下面挂的子插件一起按栈的顺序拆干净，不需要单独维护一张"谁依赖谁"的清理表。

这个机制还有两个细节保证可靠性。一是幂等：`dispose()` 首次调用把 armed 标志置 false，之后再次调用直接返回——每个逆最多执行一次。"执行两次会在从未产生过该效应的状态上应用逆，无从还原。"二是中断：如果 callback 返回的是迭代器，每一步之前会查 guard，一旦失效立即停止，只执行已累积的逆——热重载、卸载中途失败都能安全停住。事件监听器也遵循同样的可逆机制：`ctx.on(...)` 返回 unsubscribe 函数，监听器登记在 fiber 的 hook 表里，fiber 卸载时统一移除。`_unload()` 跑完所有撤销函数后，如果发现依赖又重新满足了，会立刻调 `_reload()` 重新拉起来——"换掉某个服务的实现"在运行时层面就是一次自动的卸载再加载。

一个插件的写法大致是这样（在UI界面上添加一个时钟）：




```
// 会话日志插件：注册一个事件监听 + 一个定时器，卸载时自动清理
export const name = 'session-logger'
```

```
export function apply(ctx: Context) {
  // effect 回调在注册时立即执行
  ctx.effect(() => {
    // 副作用 1：订阅事件总线，把事件追加进日志
    const off = ctx.on('tool/result', (event) => appendToLog(event))

    // 副作用 2：开一个定时器，定期把缓冲刷到磁盘
    const timer = setInterval(flushBuffer, 1000)

    // 返回撤销函数：卸载时按注册的相反顺序执行
    return () => {
      clearInterval(timer) // 后注册，先撤销
      off()                // 再退订监听
    }
  })
}
```

挂载这个插件时，`ctx.effect` 的回调立即执行：监听器挂上、定时器启动，返回的撤销函数被推进 disposables 列表。卸载时（插件被移除、会话结束、HMR 重载），`disposables.splice(0).reverse()` 按逆序逐个执行——先 `clearInterval`，再 `off()`。撤销逻辑写在注册逻辑旁边，作者不需要在别处维护一份"哪里注销"的清单。

**这个写法真的很像 React。** `useEffect` 的形状几乎一样：`useEffect(() => { subscribe(); return () => unsubscribe() }, deps)`——回调里做副作用，返回清理函数，组件卸载时自动调用。两边共享同一个核心约定：**副作用和它的逆写在一起，销毁由框架触发**。差异也清楚：React 的 effect 在渲染后执行，依赖变化时先清理再重建；Cordis 的 effect 在注册时立即执行，卸载时统一撤销，清理顺序有明确的 LIFO 保证（后注册的先撤销，子插件随父插件逆序级联）。

那"逆转"为什么值得花这么大篇幅？论文把"自进化 Agent Harness"列为两大动机之一，原文非常尖锐：

> 未来的 harness 会在持续服务请求的同时生成并部署对自己组件的修改……没有时间可组合性，每次自我修改都迫使完整重启，丢弃所有进程内累积状态；更糟的是，一个有缺陷的自我修改可能废掉唯一能用来恢复的那个进程。

这是"自进化"的前提：Agent 自己写插件/工具，运行时装进自己。装了坏的能回滚，而且回滚机制本身（宿主进程）不能被新组件弄坏。普通插件系统做不到这一点，因为重启宿主意味着丢失当前进程的所有状态。对照一下现有生态：VSCode 的扩展宿主无法卸载单个扩展，禁用/卸载必须重启整个宿主；Koishi 社区里，改完配置 reload daemon 是很常见的操作。Cordis 的逆转让"卸载"成为与"加载"对称的一等操作——服务不中断，其他插件不受影响。

这套机制顺带解决了两个常见工程问题。一是失败原子性：插件初始化中途抛错（配置校验失败、依赖缺失），fiber 进入 FAILED 状态并 dispose——已经应用的一半效应被逆序清

掉，不会残留部分初始化状态。二是级联清理，作者不用写卸载代码。论文 5.3 的关键论断是：

> 因为通过 context 的效应被自动追踪、逆自动复合，即使是没经验的作者，也能为一个插件的 context 介导效应获得有序清理，而不用写卸载路径——正确性从依赖每个作者的自觉，变成由抽象层一次性承担。

卸载一个插件 → 逆序级联卸载它的子插件、释放它的定时器/监听器/HTTP server/DB 连接。清理正确性从"每个作者写对 deactivate"变成"框架结构保证"。VSCode 模型在这里有一个短板：它的 deactivate 钩子和 activate 分离，"效应清理与创建分离，违背关注点局部性，完整清理难以验证"。

逆转机制的关键作用可以归纳为四点：**① 开发期 HMR**（改代码不重启、出错可事务回滚）；**② 生产期运行时插件装卸**（服务零中断）；**③ 自进化 Agent**（改自己、坏了能自救）；**④ 失败原子性与级联清理**（坏插件不残留影响、作者不用写卸载代码）。它把"卸载"从手动善后变成框架保证的结构性操作，这是"一切皆插件、插件可热插拔"能成立的前提。

## 三、四条约束与系统边界：框架怎么保证 effect 被执行

看到这里可能有人会问：如果作者把副作用写在 effect 外面，比如直接给 `globalThis.foo` 赋值，这套机制还有效吗？论文正面回答了这个问题。框架的处理分两边：一边在 API 层面约束，上下文修改只能通过 effect 完成；一边明确声明系统边界，边界外的操作不受追踪。

**框架从四个层面约束 effect 的用法。**

第一，API 入口唯一。想用框架的任何能力——事件 `ctx.on`、服务 `ctx.provide`、挂插件 `ctx.plugin`、配服务 `ctx.use`——都只有 `ctx` 这一个入口，而 `ctx` 上的每个操作内部都是 `ctx.effect` 的封装。API 设计上不存在绕过 effect 使用框架功能的路径。

第二，Context 是 Proxy。源码里就是一行：`const self = new Proxy<this>(this, ReflectService.handler)`。对 ctx 的属性 get/set 全部走代理，这也是反应性 coeffect（"访问即通知"）和 interception 的机制来源。所以 `ctx.foo = x` 这种直接赋值也会被拦截并记录。

第三，生命周期状态机。每个 fiber 有状态机，`ctx.effect()` 第一行是 `this.assertActive()`，在已卸载的 fiber 上创建 effect 会直接抛 `CordisError INACTIVE_EFFECT`。卸载之后再注册副作用会被框架拒绝，从时间上消除了这类泄漏。

第四，事务性 HMR。即使作者的逆写得有问题，新代码 import 失败（比如语法错误）时，Algorithm 10 的 backup/restore 会让整个重载回滚到旧版本，系统不会停留在部分加载的状态。这是机制层面的最后一道保障。

**边界外的操作，框架明确不追踪。** 论文 §6.1 把环境分成两半：边界内（inside）是系统能独占修改、且能恢复原状的位置，操作记录在 Γ，可以 recover；边界外（outside）二者缺一，操作表现为 idΓ，既不追踪也不恢复。注意边界按位置划分，与介质无关：私有路径的 scratch 文件、只有本系统写的内存属于边界内；公共文件、别的进程也在写的东西属于边界外。直接改全局变量、monkey-patch、写公共文件，都是边界外的操作。框架不会追踪这些操作，也不会声称能恢复它们。这是设计上明确声明的边界。

更细的一层是获取/发射两阶段：获取阶段（open/malloc/fork）在边界内、可逆；发射阶段（write/send 的数据离开系统）在边界外、不可逆——恢复只能靠 withholding（暂缓发射，直到状态确定）或 compensation（补偿动作：删除创建的文件、退还收到的款项）。补偿也按 LIFO 组合，但元理论不再保证。

工程上怎么处理边界外的资源？论文 §6.1 给了一个手段：**coeffect 通过物化外部位置来移动边界**——把所有对该位置的访问限制在一组操作内，其中每个操作都提供逆，于是原本表现为 idΓ 的操作变得可追踪、可恢复。在工程上，这就是服务（service）抽象：开发者通过 `ctx.database`、`ctx.assets` 访问这些资源，不直接接触数据库连接、文件句柄、子进程。资源生命周期被关进服务自己的 effect 里，服务提供者负责写对的逆，消费者只面对高层接口。副作用从插件代码各处集中到服务实现内部，作者义务集中到少数服务实现者身上。

还有最后一个问题：逆写得对不对，怎么判定？论文 §3.3.2 不要求"物理复原"，因为那做不到（free 不会还原堆布局，生成式名字不会再生）。恢复保证读作**观察等价**：

> 两个状态相关，当没有任何观察者能区分它们。比较行为而非表示……等价关系由 coeffect 各自携带的等价组装而成。

"逆转"的含义是观察层面的还原：恢复前后的状态，没有任何观察者能区分。这就是逆的正确性标准。堆布局没有还原也没关系，只要没有任何 coeffect 操作能察觉差异，就算恢复成功。

这一章的保证可以分层整理成一张表：

| 层次 | 谁保证 | 内容 |
| --- | --- | --- |
| 路径强制 | 框架（API 设计） | 用框架功能只能走 ctx，ctx 内操作全是 effect 封装 → 必然被追踪 |
| 时间强制 | 框架（状态机） | 已卸载 fiber 上建 effect 直接抛 INACTIVE_EFFECT |
| 属性拦截 | 框架（Proxy） | ctx 属性读写被代理拦截、可记录 |
| 模块级事务 | 框架（HMR backup/restore） | import 失败整体回滚，不进半重载态 |
| 逆的正确性 | 作者义务 | 运行时不验证 witness（g(δ)=γ），论文明说 |

| 层次 | 谁保证 | 内容 |
| --- | --- | --- |
| 边界外行为 | 无保证 | 全局变量 / 公共文件 / 发射出去的数据 = idГ |
| 边界移动 | coeffect / 服务物化 | 把外部位置封装成"可逆的服务"，访问受限、逆由服务提供 |

总结一下。框架把"必须通过 ctx.effect 修改上下文"变成唯一路径，把"逆是否正确"留给作者义务，并明确声明系统边界。工程上的落点是服务物化：副作用集中在服务实现内部，义务从"每个插件作者"收敛到"少数服务实现者"。论文还有一句很诚实的话："回调提供了逆，但该逆是否真的恢复了伴随它的效应，是组件作者的责任（obligation），而不是运行时验证的性质（property）"。运行时保证"逆会被调用"（结构性），不保证"逆写得对"（语义性）。

**顺带一个观察：这套设计在 TS/JS 里落地最直接。** 大多数编译语言的类型系统基于类，类型布局在编译期固定，无法通过动态加载新的编译结果来覆盖；加载进内存的类型信息通常也无法卸载。TS/JS 是原型系统，类型结构可以动态调整：Agent 造一个新工具，直接挂在原型上，所有子组件立即感知并调用；撤销时从原型链切断，系统状态立即恢复。Proxy 提供"逻辑上极度解耦、语法上完全透明"的动态路由——代码写下 `ctx.tool` 时，Proxy 完成动态寻址与权限检查，静态语言很难实现这一点。Module Augmentation 让插件能动态修改全局 Context 的类型定义，agent 生成的新工具，类型提示和安全检查瞬时同步给整个系统，实现"动态演化"与"类型安全"并存。另外，JS 有程序化的模块注册表，可以卸载模块并让垃圾回收回收相关对象；在像 Swift 这种语言里，彻底清理已加载的类型元数据几乎不可能。

## 四、ctx.<service> 怎么解析：一个 Proxy 加一条 Fiber 链

插件之间怎么互相拿到对方提供的能力，值得单独说一下，这是"everything is a plugin"能落地的另一半。

根上下文被包了一层 Proxy：`new Proxy(this, ReflectService.handler)`。访问 `ctx.tools`、`ctx.llm`、`ctx.session` 这类属性时，走的是 Proxy 的 get 陷阱。自身已有的属性直接走原生的 `Reflect.get`，遇到没见过的属性名，先查一下有没有注册过的访问器，没有就沿着当前 Fiber 往上找：

```
let fiber = ctx.fiber
while (true) {
  const impl = fiber.store?.[prop]
  if (impl) return impl.value
  if (prop in fiber.inject) throw new Error('service not active')
  if (!fiber.runtime) throw new Error('unknown service')
  fiber = fiber.parent.fiber
}
```

一路往父 Fiber 走，直到找到提供这个服务的那一层，或者确认真的没人提供，抛错。

真正把一个服务"发布"出去的地方是 `Service` 这个基类，构造时调用 `self.ctx.reflect.provide(name, self, check)`。`provide()` 本身也包在 `ctx.fiber.effect(...)` 里：往当前 Fiber 的 store 里塞一条 `{name, value, fiber, check}` 记录，再唤醒等着这个服务的依赖方。返回的撤销函数负责把这条记录从 store 里摘掉，并重新通知一遍。一个服务被移除，走的是跟插件卸载完全一样的那套 effect 撤销机制，没有另开一条特殊路径。系统里没有一个专门的"服务注册中心"类。`ctx.<name>` 这种写法背后就是一次 Proxy 拦截加一次沿 Fiber 链的查找。谁在哪一层注册了什么，直接决定了谁能看到什么。

这套设计带来一个直接后果：作用域隔离是天然的。子 Fiber 能看到父 Fiber 注册的服务，反过来不行。想让某个插件的服务只在特定范围内生效（比如某个 preset 专属的文件系统实现），不需要额外的可见性控制逻辑，把这个插件挂在对应层级的 Fiber 下面，查找链条自然会把它限制在那个范围里。

这套机制的价值在一个真实场景里能看清楚。DeepSeek Harness 定义了一份"llm"服务接口（`packages/llm/llm`），只规定"怎么发一次流式调用、怎么处理重试"，不关心具体是哪家模型。系统里同时挂着两个实现这份接口的插件：`dsh-llm-deepseek`，专门适配自家模型；`dsh-llm-pi-ai`，内部包了一个第三方 npm 库 `@earendil-works/pi-ai`，专门做多家模型厂商的协议转换（这个"pi"和外界拿来对比的那个 Pi agent harness 是两个不相关的东西，撞了名字）。官方近 40 家可选模型厂商，大部分由后面这个插件支持。两个插件互不知道对方存在，装哪个、换哪个，上层 agent loop 的调用方式不用改一行，因为它调的是接口本身。官方文档管这种搭配叫"设计验证双胞胎"：两套完全独立的实现都能满足同一份定义，从侧面证明这份接口写得足够通用。

## 五、Agent Loop 里真正值得说的几处设计

Loop 的整体形态跟眼下大多数框架差不多，模型输入输出、检测工具调用、执行、回填，不重复讲。挑几处具体做法值得展开。

第一，轮次的结束条件有多种来源。除了"没有新的工具调用"，工具执行结果本身可以带一个 `concludesTurn: true` 标记。工具的执行逻辑用这个标记声明"这一轮该停了"，不需要等模型再说一句"好了"。

第二，`max-tokens` 状态是粘性的：

```
if (turnEnds === null || turnEnds.kind !== 'max-tokens') turnEnds = stepEnd
```

一旦某个 step 因为撑满输出上限被截断，即便后续的 step 正常完成，轮次最终报告的结束原因也不会被覆盖为"正常完成"。上层（统计、日志展示）看到的"这轮被截断过"的信号，不会因为后续步骤正常完成而被覆盖。

第三，轮次真正关闭前会广播一个 `agent/turn-stopping` 事件，这是一个可以被插件截住的钩子。插件调用 `agent.steer(...)` 塞一条新消息，轮次会继续跑下去；什么都不做，默认就真的结束。"什么时候真正收尾"这件事从 loop 内部移到了外部，任何插件都能

在最后一刻决定"还没完，我还要说点什么"，loop 本身不需要预先知道有多少种"还没完"的理由。

第四，工具调度这块，每次实际调用前都会向工具询问能否并行，判断不依赖静态白名单：

```
executionMode(exec): ToolExecutionMode {
  const tool = this.resolveExecution(exec.name, exec.agent)
  if (!tool?.isConcurrencySafe) return { kind: 'exclusive' }
  return tool.isConcurrencySafe(exec.arguments) === true
    ? { kind: 'parallel' }
    : { kind: 'exclusive' }
}
```

同一个工具，不同的参数组合能给出不同的答案（读文件永远并行安全，写文件只有目标路径不冲突才安全）。调度器组队执行时，会在启动下一个调用之前重新判定一次模式。并行组里如果混进了一个当下判定为互斥的调用，会就此截断这一组，不会把互斥调用硬塞进并行批次。结果按模型原始给出的调用顺序提交，与调度完成顺序无关。

第五，工具执行前后的检查是独立的、可插拔的阶段，各自单独注册，不写在工具执行函数内部。`tools/pre-execute` 是一个 waterfall 事件，任何插件都能在这里拦截或者要求审批；通过后才进入内置的防护检查（比如同一个工具带同样参数被连续调用太多次会被提醒）；然后才是真正调用工具体；跑完还有 `tools/post-execute` 让插件对结果做二次处理。这四段是固定顺序、彼此独立注册的插件挂载点。新增一种审批策略或者防护规则，不需要改动执行链路本身，挂一个新的事件监听就行。

## 六、Preset 的两层 Scope 链：真实继承与影子路由表

Preset 的挂载机制解决了一个具体问题：怎么让同一份配置被多个会话复用，又不用每次都重新解析一遍配置文件。

一份预设是一个目录，核心是一份 `agent.cordis.yml`，按顺序列出要挂载的插件和参数。挂载这份文件用的是一个复用自 cordis 生态的 `Include` 插件，改名叫 `PresetTree`：

```
const handle = agentCtx.plugin(PresetTree, { path: pathToFileURL(preset.path).href })
await handle.await()
```

`PresetTree` 重写了两个方法：解析裸模块名时，查找基于系统自己的 baseUrl，预设目录不参与解析；写回配置被直接禁用，预设源文件不会被意外修改。挂载完成后还会核对两件事：有没有哪一行插件配置停在不可用的状态，有没有哪个服务在挂载过程中"泄漏"到预设作用域之外。任何一项不满足，都会整体回滚。

scope 链的构造方式值得细看。从全局到某个预设，是一次真实的上下文派生：`createScope()` 内部调 `ctx.plugin(scope)` 拿到一个 Fiber，再 `fiber.ctx.extend({[kScope]: key})`，这一步在 Cordis 的上下文树里真实存在一个节点。从预设到具体某个会话，则使用另一种方式：

```
this.bindings.set(agentKey, bindScopeParent(agentKey, standing.key))
```

`bindScopeParent` 往一张 WeakMap 里记一条"逻辑上级"关系，不在 Cordis 的 Fiber 树里新开任何节点。

这么拆开的动机很直接：一份预设的"标准挂载实例"只建一次（`ensureStanding()`），后续任意多个会话——写作的、编码的、调研的——都可以复用同一份实例，靠的就是往这张 WeakMap 里追加一条绑定，不用每接一个新会话就重新跑一遍插件加载。子 agent 要继承父 agent 当前挂的预设，调用的 `composeFrom(childCtx, parentCtx)` 也只是这张表上的一次查找加绑定，是同步操作，不会重新触发任何插件的挂载流程。子 agent 拿到的是跟父 agent 完全同一份插件实例，包括同一批已经注册好的工具和提示词片段。

会话当前生效的是哪个预设，判定逻辑是倒着扫事件日志，找最新一条 `agent-preset/selected`：

```
for (let index = session.events.length - 1; index >= 0; index -= 1) {
  const event = session.events[index]
  if (event?.type === 'agent-preset/selected') return event.data.agentPreset
}
return session.header.agentPreset
```

完全没有这类事件才回退到会话创建时的默认值。理论上，一个会话中途换预设是架构允许的操作，查找逻辑天然支持这种情况，实际用不用是产品层面的决定。

预设的发现机制也值得一提。扫描一个预设根目录时，只认目录名匹配特定命名规则、且内部有 `agent.cordis.yml` 的子目录。逐个校验 YAML 是否能解析、结构是否合法，损坏的条目标记出来，不会中断整个扫描。多个根目录合并时，按顺序先出现的 id 优先，后面根目录里同名的预设被忽略——这给了"用户自定义预设覆盖官方预设"（或反过来）的空间，取决于根目录的注册顺序。

## 七、Code Mode 的隔离方案：worker_threads

让模型写代码去编排多个工具调用，这个思路眼下不算新鲜，Codex 也做了一套。值得细看的是隔离方案的选型和落地方式。

DeepSeek Harness 跑这段代码用的是 `node:worker_threads`，一个普通的 Node 工作线程，与常见的 V8 隔离环境或者 `vm2` 是不同类型的方案。选这条路有一个明确考量：`node:vm` 那类基于同进程的沙箱，原型链可以逃逸到主环境，热循环也无法从外部真正打断，被认为不可靠。换成独立的工作线程，代价是每次跑代码都要开一个新线程，换来的是一个真正独立的 V8 堆，以及一个可以从外部强制终止的执行环境。

模型写的代码，先过一遍 `stripTypeScriptTypes` 把类型标注剥掉，剥的时候用一层包装前后缀撑住位置：

```
const stripped = stripTypeScriptTypes(prefix + program + suffix)
```

保证剥完之后代码原本的行列号不变，报错定位对得上。剥完的代码被塞进一个动态构造的异步函数：

```
const AsyncFunction = (async () => {}).constructor
const fn = new AsyncFunction(...bindingNames, ...errorClassNames, 'console', code)
const value = await fn(...bindings, ...errorClasses, consoleShim)
```

每个工具命名空间、每个约定好的错误类、还有一个替换用的 console，都变成这个函数的形参和实参。模型写的代码里可以直接用顶层 `await`，也能直接 `return` 一个值出来。

工作线程本身有资源上限，`resourceLimits.maxOldGenerationSizeMb` 限制堆内存。另外有两道独立的超时：一道拿事件循环利用率轮询，计算这段代码占了多少 CPU 时间；一道是一个简单的 `setTimeout`，兜底总耗时。任意一道触发都会强制中断。

代码里调用工具，通过消息通道完成，不直接引用函数：

```
{ type: 'call', id, global: 'tools', name, args }   // worker → host
{ type: 'reply', id, ok: true, value }              // host → worker
```

宿主收到调用请求后，把它当成一次正常的工具调用，推进第五章讲的那条 pre-execute/防护/dispatch/post-execute 流水线，与模型直接发起的结构化工具调用共用同一套执行内核，只是入口不同。通道两端都手动校验每一条收到的消息字段。宿主侧查找绑定函数用 `Object.hasOwn()` 精确判断自有属性，构造暴露给代码的 `tools` 命名空间时，用 `Object.create(null)` 起手，再逐个 `Object.defineProperty` 挂载。这些做法都是为了同一个目的：防止代码里塞进 `__proto__` 或 `constructor` 这类字段，绕过原型链产生意料之外的行为。代码注释里直接写"把 worker 当成敌对的另一方来处理"，这个信任假设是明确声明的。

Codex 那套做法的核心差异在隔离层：用一个独立的 V8 isolate，工具挂在一个全局的 `tools` 对象上，官方描述是"在一个全新的 V8 隔离环境里执行这段 JavaScript"。两边解决的是同一个问题——用代码编排多个工具调用、减少往返——隔离技术选型不同，背后权衡的是"每次新建一个 isolate 的开销"和"每次开一个工作线程的开销"哪个更划算，以及各自生态里现成的隔离原语是什么。

## 八、工具注册的分层遮蔽算法

工具怎么在不同作用域之间"就近覆盖"，是插件化设计里容易被忽略但很关键的一块，决定了 preset 能否真正替换工具。

每个作用域维护一份自己的 `ToolLayer`，内部是一份按名字索引的表。决定"当前 agent 能看到哪些工具"的，是 `view()` 这一步：先加载全局层的工具，作为基础；再让每一层祖先 scope 依次覆盖同名条目，越靠近当前 scope 的祖先优先级越高——这一步只处理继承来的那部分。限制规则（比如某个 preset 明确禁掉了某个工具）在这之后筛一遍。最后，当前 scope 自己直接注册的工具，覆盖前面所有继承来的同名条目，即使限制规则禁用了这个名字。这一步的顺序很关键：自己注册的工具优先于继承和限制规则。

注册这一步本身也有硬校验：

```
register(definition: ToolDefinition): () => void {
  if (definition.name === RUN_CODE_NAME) {
    throw new Error(`tool name "${RUN_CODE_NAME}" is reserved for the Code Mode trans
  }
  assertSupportedJsonSchema(definition.output.schema)
  return this.layers.effect(this.ctx, layer => layer.tools.insert(definition.name, de
}
```

`run_code` 这个名字被硬编码保留，任何插件都不能注册或者覆盖它。理由写在代码注释里：任何 agent 都可能给自己选一个代码模式，一个在默认部署下看似空闲的名字，一旦某个 preset 挂载进来，随时可能跟 Code Mode 的调用入口产生冲突，所以无条件预留。

注册动作本身也走 `effect()`：调用 `register()` 拿到的撤销函数，插进当前 Fiber 的 disposables 列表。一个插件注册的工具，会随着这个插件被卸载自动从层里摘掉，不需要额外写一段"卸载时记得把工具删掉"的清理代码——这跟第二章讲的效果撤销机制是同一条路径，工具注册只是众多"注册即副作用"里的一种。

工具执行本身也复用了这套调度器的并发契约。Code Mode 里模型代码调用工具时，调用会进入宿主侧同一个 `TOOL_RUNTIME_SCHEDULER` 的待处理队列，与模型直接发起的结构化工具调用共享同一份并发控制、同一套前置检查——这也是上一章说 Code Mode 与普通工具调用"共用同一套执行内核，只是入口不同"的原因。

## 九、跟 Codex 比，几个工程决策的差异

**核心循环的可替换性。** Codex 的工具接口、审批策略、沙箱策略都能配置，但驱动"采样-执行-反馈"这条循环的代码固定在核心中，无法作为独立单元替换。DeepSeek Harness 里这条循环的具体实现（`ReactLoopAgent`）注册为一个工厂，理论上可以被另一套完全不同的循环实现替换掉。目前只有这一个实现挂在系统里，工厂只允许注册一次。

**沙箱的位置。** Codex 把 Linux 下的 bwrap/Landlock、macOS 下的 Seatbelt，直接编译进核心执行路径，属于基础设施。DeepSeek Harness 把沙箱做成一层能力插件，Linux 上通过一个独立的原生 Node 扩展调用 Landlock，走的是前面几章讲的同一条服务注册路径，理论上可以换成别的隔离实现，调用它的工具代码不需要改动。

**代码规模的拆分粒度。** Codex 的 Rust 代码分成大约上百个 crate；DeepSeek Harness 的 TypeScript 代码分成两百多个包。两边都不是单体架构，但 Codex 拆出来的模块最终都服务于同一个不可替换的核心循环；DeepSeek Harness 拆出来的模块（包括循环本身）理论上都处在同一个可替换层级上，没有哪一个模块的地位特殊。

**代码编排工具调用的隔离选型。** 两边独立得出了同一个功能判断：工具调用除了单个结构化请求之外，还应该支持用代码编排。落地时隔离技术不同：Codex 选了独立的 V8 isolate，DeepSeek Harness 选了 worker_threads。第七章已经展开过这个差异背后的取舍。

**工具暴露方式的粒度。** Codex 的工具接口里，暴露方式（`DIRECT` / `DEFERRED` / `CODE_MODE` / `Hidden`）是工具自己声明的一个静态标记。

DeepSeek Harness 里，一个工具在某个 scope 下可见还是被覆盖，是运行时按 scope 链实时计算出来的，同一个工具在不同会话里的可见性可以完全不同，取决于挂载了哪些 preset。

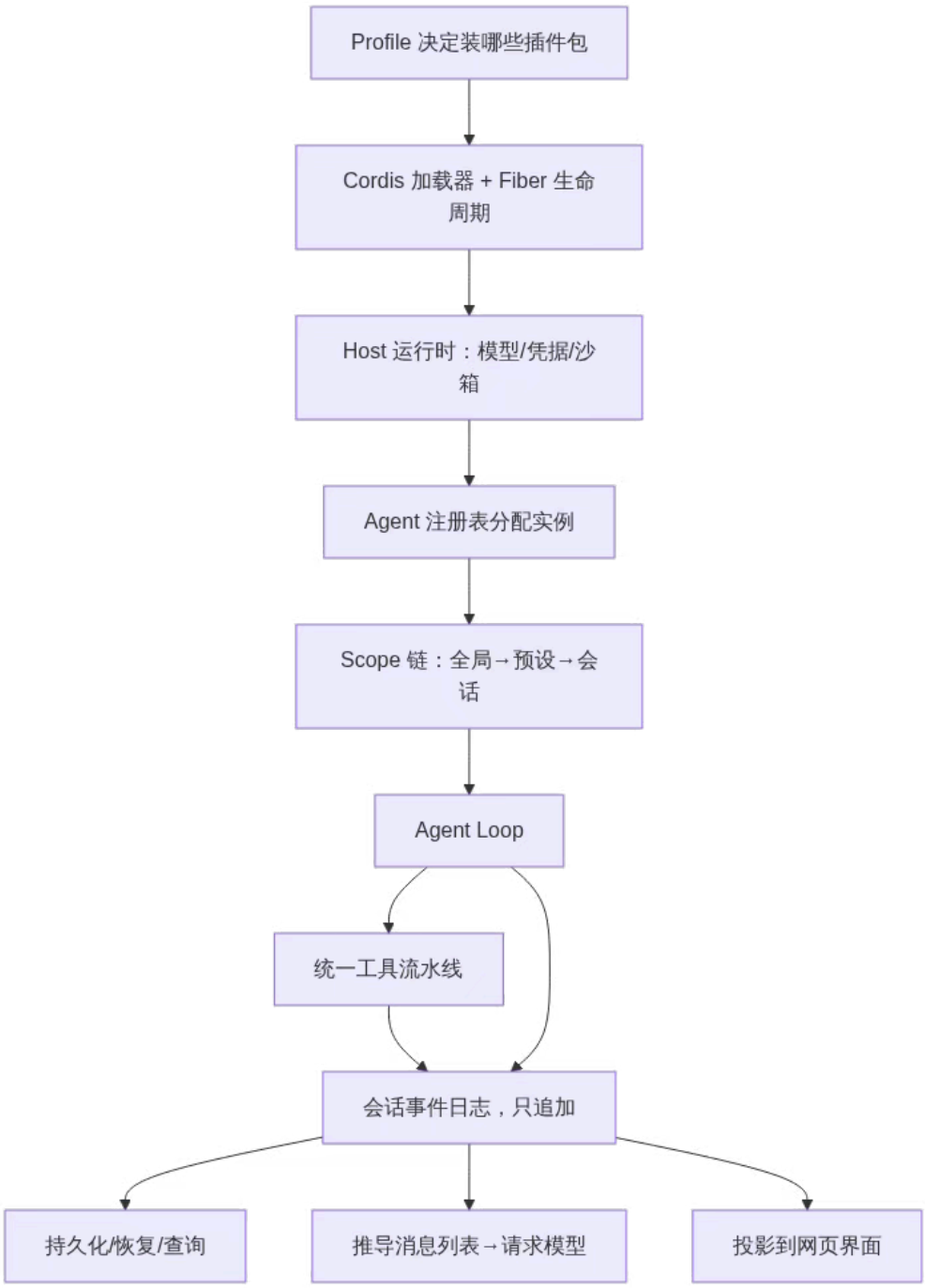
## 附：从进程启动到一次工具调用，整条链路走一遍

进程启动，先看用的是哪个启动档（Profile），决定整个进程要装哪些插件包，交给 Cordis 的加载器逐个挂载，每个插件对应一个 Fiber。装完，host 这一层准备好模型接入、凭据管理、沙箱这些运行时基础设施。

一个会话开始，agent 注册表分配一个具体实例，这个实例挂在"全局-预设-会话"这条 scope 链下面。能看到哪些工具、哪些提示词，取决于它在这条链上的位置，以及第八章讲的那套遮蔽规则。

Agent Loop 接管这个会话的运转：读取当前上下文，请求模型，检测工具调用，交给统一的工具流水线走一遍前置检查、执行、后置处理。

每一步发生的事，不管是模型的输出还是工具的结果，都作为一条事件追加进这个会话的日志里，不删除、不覆盖。这份日志同时是持久化的来源、下一次请求的上下文来源、网页界面的展示来源，三条用途都从同一份日志投影出来。网页界面本身也是挂在这套体系里的一组插件。




这篇文章讲了不少具体机制：Cordis 的 Fiber 生命周期和 effect 撤销栈，`ctx.<service>` 的 Proxy 解析，agent loop 里几处细节设计，preset 的两层 scope 链，Code Mode 的隔离选型，工具注册的分层遮蔽算法。这些机制拼在一起，解决的是做 agent 产品、或者深度使用 agent 时会反复遇到的几个具体问题。

第一个问题是加能力和改能力的风险。常见的做法是在一个核心循环外面挂插件，核心本身很少改动，因为改动一次要担心影响面，回归测试也要重新跑。DeepSeek Harness 把风险控制粒度做到了插件级：一个插件注册的所有东西都通过 effect 登记，卸载时按注册的反顺序原样撤销。新增或者替换一个能力，影响范围被框定在这个插件自己的边界里，验证范围也缩小到这个边界。

第二个问题是，想让同一套 agent 服务不同场景，又不想每次都重新搭一遍环境。preset 的两层 scope 链解决的是这个：一份预设的插件树只挂载一次，写作、编码、调研几种不同用途的会话共享同一份已经跑起来的实例，靠一张逻辑父子表分流复用。

第三个问题是多步骤任务里，一次一个工具调用的来回成本。Code Mode 把这类任务的编排逻辑交给模型自己写代码解决，减少了"说一句、等一下"的往返次数。这块 DeepSeek Harness 和 Codex 几乎同时给出了同一个判断，只是隔离方案选得不一样。

第四个问题是工具的可见性和权限，在不同场景下常常只有两种状态：一个工具要么全局可用，要么全局不可用。分层遮蔽算法把可见性变成运行时按 scope 链实时计算的结果，同一个工具在不同预设、不同会话里可以有完全不同的可见性，工具代码里不需要写一堆条件判断。

这几处设计共同指向一个更大的判断：agent 产品的竞争力，除了模型效果，还取决于运行时这一层。运行时怎么组织、扩展成本有多高、换一个组件要付出多大代价，这些工程决策同样会影响一个 agent 产品用起来顺不顺手。DeepSeek Harness 把重点放在这一层，用 Cordis 这套现成的插件框架，以及"可逆副作用"这套设计哲学，把这一判断落实到具体的代码结构上。效果怎么样，还得看这套架构在真实使用里能撑多久，能不能撑住更复杂的场景。

> P.S. 在写这篇文章中提到的那个示例插件的时候还把UI给搞崩了，看来HMR的稳定性还有待提升啊
>
> 
> 

## 展望：一个能对自己热更新的运行时，会走到哪

社区里已经有人给 DSH 做了连接通讯工具的插件了，比如把 Telegram、Discord 上的消息接进来。当年 Koishi 靠 Cordis 把 QQ、Discord、Telegram、微信的几十个插件拼在一起，成了聊天机器人圈的现象级项目，也留下了"改完配置 reload daemon"的日常。如今同一个运行时内核被搬进 agent 产品，通讯工具插件开始出现——当年那条路正在 DSH 上重演，只是这一回，底层自带一条 agent loop。

另一条路也在同时进行。Hermes、OpenClaw 这类个人 AI 助手的成长方式，是往技能库里不断加 skill：能力变多，运行时的结构基本不动。DSH 的成长方式在另一个维度：插件可以热装卸，界面是插件、会话记录是插件、循环本身是插件，连 webui 都能在持续服务的同时被换掉。论文里那句"未来的 harness 会在持续服务请求的同时生成并部署对自己组件的修改"，在这件事上已经开始兑现了。

所以问题值得认真问一遍：那个整天 reload daemon 的"龙虾"，动不动就把自己给搞炸，这次是不是真的可以下班了？DSH 会成为旧一代"插件机器人框架 + 手动配置"工作流的 AI

上位替代吗?

现在没有答案，但 webui 即插件，意味着产品形态本身可以热更新：改界面、换交互、加新视图，不需要重启宿主，也不需要发布新版本。要替代的如果是一种工作方式，那它就是"拼插件、写配置、重启验证"这个循环本身被压缩了。

对个人开发者来说，想做一个 AI 产品，以前要写前端、配后端、搭 agent 框架，三块互不相通；在 DSH 上，这三层都是插件，挂进同一个运行时，界面还能直接热更新。一个想法到可用的原型，甚至到能交付的成品，中间的距离正在被明显压缩。原型阶段跑通的东西，加一个 preset、配一个插件，就可能直接变成产品的一部分。

不过，从"偏开发者"走到"面向普通开发者"，中间还有一段路。现在跑 DSH 需要装 Node 环境，开终端手动敲命令启动 server。这套用法对写代码的人是日常，对普通开发者来说确实算不上优雅。技术复杂度不降下来，"个人快速做出 AI 产品"就只对少数人成立。好在插件化的好处恰恰在这里：宿主怎么启动、界面怎么呈现，都是可以替换的插件。把这层复杂度包进安装包、包进一键启动，属于产品化的问题，架构上已经有位置放它了。

它能不能成为这个家族里下一代的现象级项目，最后取决于几件事：生态里长出来的插件够不够多，真实场景撑不撑得住，面向普通开发者那层使用复杂度什么时候能被包起来。

目前 DeepSeek Harness 还是一个偏向开发者的产品，但相信距离其发展成一个“Agent 界的 VSCode”这一天已经不远了。

## 推荐插件：生态建设飞快！

这篇文章写到这里，GitHub 上 #dsh-plugin 这个 topic 下已经有 1000+ 个仓库了（检索时 API 统计 1008 个）。官方本体 deepseek-ai/deepseek-harness 之外，第三方插件、客户端、教程都在长。要翻全量清单，从几个精选索引入手最快：

- awesome-dsh-plugin/awesome-dsh-plugin（256⭐）— 插件精选列表，中英双语
- AdamPlatin123/awesome-dsh-plugins（507⭐）— 自动扫描索引所有 dsh 插件候选
- 0xsline/awesome-deepseek-harness（227⭐）— DSH 生态精选：插件、工具、基础设施
- Electricitysheep/dsh-handbook（74⭐）— 从 0 到 1 的 DSH 深度手册，含插件开发

**Web UI 增强**：

- zhu1090093659/dsh-web-ui（922⭐）— 任务面板、Git 图、皮肤中心合集
- omdsh-dev/DSH-better-sidebar（371⭐）— 侧边栏工作台：文件、终端、Git、子代理
- Small-tailqwq/dsh-deep-whale（237⭐）— 鲸鱼娘皮肤系列
- lhh010/dsh-minigames（10⭐）— 右侧 18 款小游戏摸鱼面板

**视觉 / 多模态**

- liustack/modlens（857⭐）— 第一个视觉插件：OCR、布局、语义结构化证据
- Anionex/dsh-vision-toolkit（232⭐）— 图片问答、长截图 OCR、UI 还原

**终端与桌面端——"把复杂度包进安装包"已经有社区在做**：

- ccch1mneyyy/dsh-TUI（487⭐）— Claude Code 风格全屏终端 UI
- Ruler4396/dsh-launcher（40⭐）— Windows 轻量启动器
- bruc3van/dsh-desktop（10⭐）— 社区第三方桌面客户端

**记忆 / 上下文——让会话跨过 session 的边界。**

- csyangwen/dsh-memory-evolve（24⭐）— 跨会话长期记忆 + 自我进化
- Anionex/dsh-turn-rewind（25⭐）— 对话 / 代码状态回退

**多 Agent / 工作流——把 agent 编排成团队。**

- NanmiCoder/dsh-agent-teams（142⭐）— AgentTeams 插件
- icetomoyo/dsh_workflow（42⭐）— 把一次性调度升级为可治理的 Workflow 层

**通讯与通知**

- PlutoKeating/dsh-lark-bot（4⭐）— 飞书机器人
- sliverp/DeepSeek-harness-qqbot — QQ 机器人
- LoserFox/telegram — Telegram
- omdsh-dev/dsh-notification（26⭐）— 桌面通知

**开发配套——做插件的人先给自己做了工具。**

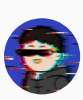
- omdsh-dev/dsh-genui（36⭐）— 对话内生成式 UI
- omdsh-dev/dsh-at-file（62⭐）— Codex 风格 @file 引用
- vlln/plugin-registry（21⭐）— 插件生态基建 + 开发引导

star数为写文时参考值，具体以实际为准。完整清单，可以查看 github.com/topics/dsh-plugin 。

撰文的参考资料，感谢大佬们分析：

- 为什么用 Cordis 做 AI Agent 运行时：从 QQ 机器人框架到 DeepSeek Harness（CSDN）
- DeepSeek Harness 背后的秘密，Cordis 的设计哲学深度解读（知乎）
- Cordis 88 页设计论文：一个运行中的 Agent，能不能"换零件"而不重启？

今晚19:00直播DeepSeek Harness 拆解，不见不散～



**腾讯程序员**

已结束直播，可观看回放

观看回放

DeepSeek Harness







阅读 5.2万

**留言**

写留言

还没有留言，来写首评